

Describing the UI: Hinglish Explanation

Ye chapter React ke **UI banane wale foundational concepts** ka overview hai. Iska central idea ye hai:

Screen par dikhne wale interface ko chhote, reusable **components** mein divide karo, phir un components ko combine karke complete application banao.

Button, image, heading, card, navigation bar aur poora page, ye sab React components ho sakte hain.

1. React component kya hota hai?

React application isolated UI pieces se banti hai, jinhe **components** kehte hain.

Technically, React component ek **JavaScript function** hota hai jo JSX markup return karta hai:

```
1 function Profile() {  
2   return (  
3     profile.jpg  
4   );  
5 }
```

Is component ko dusre component ke andar HTML tag ki tarah use kar sakte hain:

```
1 function Gallery() {  
2   return (  
3     <section>  
4       <h1>Amazing scientists</h1>  
5  
6       <Profile />  
7       <Profile />  
8       <Profile />  
9     </section>  
10  );  
11 }
```

Yahan kya ho raha hai?

- Profile ek reusable component hai.
- <Profile /> likhne par React us component ko render karta hai.
- Same component ko teen baar use kiya gaya hai.
- Gallery parent component hai.

- Profile child component hai.

Important rule

React component ka naam **capital letter** se start hona chahiye:

```
1 function Profile() {}
```

Lowercase names jaise <profile /> ko React normally built-in HTML element samajh sakta hai.

2. Components ko import aur export karna

Chhoti application mein multiple components ek file mein rakh sakte ho. Lekin application badi hone par file ko navigate aur maintain karna difficult ho jata hai.

Isliye components ko separate files mein divide kiya jata hai.

Profile.js

```
1 export default function Profile() {  
2   return profile.jpg;  
3 }
```

Gallery.js

```
1 import Profile from './Profile.js';  
2  
3 export default function Gallery() {  
4   return (  
5     <section>  
6       <h1>Amazing scientists</h1>  
7       <Profile />  
8       <Profile />  
9       <Profile />  
10      </section>  
11    );  
12  }
```

App.js

```
1 import Gallery from './Gallery.js';  
2  
3 export default function App() {  
4   return <Gallery />;  
5 }
```

Isse dependency kuch aisi banti hai:

```
1 App
2   └─ Gallery
3       └─ Profile
4       └─ Profile
5       └─ Profile
```

Basic meaning

- export default component ko file ke bahar available banata hai.
- import component ko doosri file mein laata hai.
- './Profile.js' ka matlab current folder ki Profile.js file hai.

Normally ek file ka primary component default export kiya ja sakta hai.

3. JSX kya hai?

JSX ek syntax extension hai jo JavaScript file ke andar HTML-jaisa markup likhne deta hai.

```
1 function Heading() {
2   return <h1>Hello React</h1>;
3 }
```

Ye HTML jaisa dikhta hai, lekin exactly HTML nahi hai. JSX ke rules thode strict hain.

Rule 1: Ek parent element return karo

Ye invalid hai:

```
1 return (
2   <h1>Title</h1>
3   <p>Description</p>
4 );
```

Dono elements ko ek parent ke andar wrap karna hoga:

```
1 return (
2   <div>
3     <h1>Title</h1>
4     <p>Description</p>
5   </div>
6 );
```

Agar extra div nahi chahiye, to **Fragment** use karo:

```
1 return (
2   <>
```

```
3     <h1>Title</h1>
4     <p>Description</p>
5     </>
6 );
```

<>...</> ko Fragment syntax kehte hain.

Rule 2: Saare tags close karo

HTML mein kabhi-kabhi closing tags optional lag sakte hain, lekin JSX mein elements properly close hone chahiye:

```
1 photo.jpg
1 <li>First item</li>
```

Rule 3: class ki jagah className

HTML:

JSX:

```
1 <img className="photo" />
```

Reason ye hai ki JSX JavaScript ke close hai aur class JavaScript ka reserved keyword hai.

4. JSX ke andar JavaScript use karna

JSX ke andar dynamic JavaScript values use karne ke liye curly braces {} lagate hain.

```
1 const person = {
2   name: 'Gregorio Y. Zara'
3 };
4
5 function Profile() {
6   return <h1>{person.name}'s Todos</h1>;
7 }
```

Output:

```
1 Gregorio Y. Zara's Todos
```

Curly braces ko JSX se JavaScript ki duniya mein jaane wali **window** samajh sakte ho.

Attributes mein JavaScript

```
1 {person.imageUrl}
```

Yahan quotes nahi lagaye gaye, kyunki values JavaScript expressions hain.

Inline styles

```
1 const theme = {
```

```

2   backgroundColor: 'black',
3   color: 'pink'
4 };
5
6 function Page() {
7   return <div style={theme}>Content</div>;
8 }

```

style ko CSS string nahi, JavaScript object diya jata hai.

CSS property names camelCase mein hoti hain:

```
1 backgroundColor
```

na ki:

```
1 background-color
```

5. Props se components configure karna

Props parent component se child component ko information pass karne ka method hain.

Ye HTML attributes jaise dikhte hain:

```

1 <Avatar
2   person={person}
3   size={100}
4 />

```

Child component props ko receive kar sakta hai:

```

1 function Avatar({ person, size }) {
2   return (
3     {person.imageUrl}
4   );
5 }

```

Yahan { person, size } object destructuring hai.

Conceptually React component ko aise call samajh sakte ho:

```

1 Avatar({
2   person: person,
3   size: 100
4 });

```

Props ke through aap pass kar sakte ho:

- Strings
- Numbers

- Booleans
- Objects
- Arrays
- Functions
- JSX

children prop

Agar component ke opening aur closing tags ke beech content pass karte ho:

```
1 <Card>
2   <Avatar />
3 </Card>
```

To Card ko wo content children prop mein milta hai:

```
1 function Card({ children }) {
2   return (
3     <div className="card">
4       {children}
5     </div>
6   );
7 }
```

children wrapper components ke liye useful hai, jaise:

- Card
- Modal
- Layout
- Panel
- Sidebar section

6. Conditional rendering

Kabhi UI ko kisi condition ke according display karna hota hai.

React mein iske liye normal JavaScript techniques use ki jaati hain:

- if
- ternary operator ? :
- logical AND operator &&

&& operator

```
1 function Item({ name, isPacked }) {
2   return (
```

```
3     <li>
4         {name} {isPacked && '☑'}
5     </li>
6 );
7 }
```

Agar isPacked true hai, to checkmark render hoga.

```
1 <Item name="Space suit" isPacked={true} />
```

Output:

```
1 Space suit ☑
```

Agar false hai:

```
1 <Item name="Photo" isPacked={false} />
```

Output:

```
1 Photo
```

Ternary operator

Jab true aur false dono cases mein alag content dikhana ho:

```
1 function Item({ name, isPacked }) {
2     return (
3         <li>
4             {isPacked ? `${name} ☑` : `${name} ✕`}
5         </li>
6     );
7 }
```

Simple decision

- Sirf true hone par kuch dikhana hai: &&
 - True aur false dono ke liye output hai: ? :
 - Complex logic hai: if
-

7. Lists render karna

Jab array ke har item ke liye similar UI banana ho, tab JavaScript ka map() use karte hain.

```
1 const people = [
2     { id: 1, name: 'Aman' },
3     { id: 2, name: 'Riya' },
4     { id: 3, name: 'Kabir' }
```

```

5 ];
6
7 function PeopleList() {
8   const listItems = people.map(person => (
9     <li key={person.id}>
10       {person.name}
11     </li>
12   ));
13
14   return <ul>{listItems}</ul>;
15 }

```

map() array ke har person ko ek JSX element mein transform kar raha hai.

Conceptually:

```

1 Data array
2   ↓ map()
3 JSX elements ka array
4   ↓
5 Rendered list

```

filter() ka use

Pehle data filter karke phir render kar sakte ho:

```

1 const chemists = people.filter(
2   person => person.profession === 'chemist'
3 );
4
5 const listItems = chemists.map(person => (
6   <li key={person.id}>
7     {person.name}
8   </li>
9 ));

```

8. key prop kyun zaroori hai?

List render karte waqt har JSX item ko unique key deni chahiye:

```

1 <li key={person.id}>
2   {person.name}
3 </li>

```

key React ko batati hai ki list ka kaunsa element kis data item se connected hai.

Ye especially important hai jab list:

- Reorder hoti hai
- Items add hote hain
- Items delete hote hain
- Items update hote hain

Normally database ID ko key banana best hota hai:

```
1 key={person.id}
```

Key ko siblings ke beech unique hona chahiye. Puri application mein globally unique hona zaroori nahi hai.

9. Components pure rakhna

React components ko **pure functions** ki tarah behave karna chahiye.

Pure function ke do major properties hain:

1. Function apne bahar ke existing variables ya objects ko modify nahi karti.
2. Same input dene par same output milta hai.

Impure component

```
1 let guest = 0;
2
3 function Cup() {
4   guest = guest + 1;
5   return <h2>Tea cup for guest #{guest}</h2>;
6 }
```

Problem ye hai ki Cup render hote hi external guest variable change kar raha hai.

Is wajah se output render order ya component ke kitni baar execute hone par depend karne lagta hai. Behaviour unpredictable ho sakta hai.

Pure solution

```
1 function Cup({ guest }) {
2   return <h2>Tea cup for guest #{guest}</h2>;
3 }
4
5 function TeaSet() {
6   return (
7     <>
8     <Cup guest={1} />
9   )
10 }
```

```
9      <Cup guest={2} />
10      <Cup guest={3} />
11    </>
12  );
13 }
```

Ab Cup external variable change nahi kar raha. Uska output sirf guest prop par depend karta hai:

```
1 guest = 1 → Tea cup for guest #1
2 guest = 2 → Tea cup for guest #2
3 guest = 3 → Tea cup for guest #3
```

Pure components code ko zyada predictable aur debugging ko easier banate hain.

10. UI ko tree ki tarah samajhna

React component relationships ko tree ki form mein model karta hai.

```
1      App
2     /  \
3  Header Gallery
4         |
5       Profile
```

Isse **render tree** kaha jata hai.

Tree terminology

- Sabse upar wala component: **root component**
- Upar ke components: **top-level components**
- Jo component kisi aur component ko render kare: **parent**
- Jo parent ke andar render ho: **child**
- Jiska koi child component na ho: **leaf component**

Render tree samajhne se aapko component relationships, data flow aur rendering behaviour samajhne mein help milti hai.

11. Module dependency tree

Component tree aur module tree related zaroor hain, lekin same nahi hain.

Agar files aise import ho rahi hain:

```
1 App.js
2   └─ imports Gallery.js
```

```
3  |— imports Header.js
4  |— imports Footer.js
5
6  Gallery.js
7  |— imports Profile.js
```

To ye **module dependency tree** hai.

Difference

Render tree batata hai:

Kaunsa component UI mein kis component ko render karta hai?

Module dependency tree batata hai:

Kaunsi JavaScript file kis doosri file ko import karti hai?

Build tools dependency tree ko use karke relevant JavaScript code ko bundle karte hain. Bahut bada bundle user experience ko negatively affect kar sakta hai, isliye dependency structure performance debugging mein bhi useful hota hai.

Chapter ka complete mental model

Is poore chapter ko is flow mein samjho:

```
1  UI ko components mein divide karo
2      ↓
3  Components ko JavaScript functions ke form mein likho
4      ↓
5  JSX se unka markup describe karo
6      ↓
7  Props se components ko data do
8      ↓
9  Conditions ke basis par UI change karo
10     ↓
11  map() se data lists render karo
12     ↓
13  Components ko pure rakho
14     ↓
15  Render tree aur dependency tree samjho
```

Sabse important takeaways

1. React UI reusable **components** se banti hai.

2. Components JavaScript functions hote hain jo JSX return karte hain.
3. JSX HTML jaisa hai, lekin uske rules stricter hain.
4. {} se JSX ke andar JavaScript expressions use hote hain.
5. Props parent se child component ko data pass karti hain.
6. children se wrapper component ke andar nested JSX milta hai.
7. if, && aur ternary se conditional UI render hoti hai.
8. map() data ko JSX list mein transform karta hai.
9. Har list item ko stable aur unique key deni chahiye.
10. Components ko external values mutate kiye bina pure rakhna chahiye.
11. Render tree component relationships dikhata hai.
12. Dependency tree files aur imports ke relationships dikhata hai.

External resource mentioned on the page

- [HTML ko JSX mein convert karne ka tool:\):\)](#)